UNIVERSIDADE FEDERAL DE ALFENAS

LUIZ FERNANDO FERREIRA CABRAL

JOAQUIM PEDRO DO NASCIMENTO MOREIRA DE JESUS

VICTÓRIA DE ALMEIDA TAMBASCO

LUIZ GABRIEL DA SILVA CABRERA

MURILO ANTONIO DA SILVA

# PAREAMENTO DE SEQUÊNCIAS DE DNA

Relatório para disciplina AED's III, pela instituição Universidade Federal de Alfenas, Bacharelado em Ciência da Computação.

Professor: Iago Augusto de Carvalho.

ALFENAS
2026

## RESUMO

Este trabalho apresenta uma implementação e análise comparativa de dois paradigmas algorítmicos — **Programação Dinâmica** e **Algoritmo Guloso** — aplicados ao **Problema do Pareamento de Sequências de DNA**. O objetivo central reside em alinhar strings compostas por bases nitrogenadas (A, T, C, G) através da introdução estratégica de lacunas (*gaps*), visando a maximização da pontuação de pareamento baseada em pesos específicos para acertos, erros e lacunas. Inicialmente, descreve-se a fundamentação biológica e computacional do problema, destacando sua relevância na bioinformática como um pilar para a compreensão de instruções genéticas.

Posteriormente, a eficácia de cada método é avaliada através de experimentos práticos com diversas instâncias de entrada. A abordagem por **Programação Dinâmica** é explorada como o método capaz de garantir a otimalidade global da pontuação, embora apresente um custo computacional polinomial mais elevado. Em contrapartida, o **Algoritmo Guloso** é analisado sob a ótica da eficiência temporal, demonstrando execuções significativamente mais rápidas, contudo, sem a garantia de encontrar a pontuação máxima em todos os cenários de alinhamento.

Ao final, são discutidas as implicações dos resultados obtidos, focando no *trade-off* entre tempo de execução e otimalidade dos alinhamentos gerados. A análise crítica evidencia como diferentes paradigmas podem ser escolhidos dependendo das restrições de recursos e da precisão necessária em aplicações reais de análise genômica.



**Palavras-chave:** Bioinformática; Programação Dinâmica; Algoritmo Guloso; Pareamento de DNA; Complexidade computacional; Otimização.

## ABSTRACT

This work presents an implementation and comparative analysis of two algorithmic paradigms — **Dynamic Programming** and **Greedy Algorithm** — applied to the **DNA Sequence Alignment Problem**. The central objective is to align strings composed of nitrogenous bases (A, T, C, G) through the strategic introduction of gaps, aiming to maximize the alignment score based on specific weights for matches, mismatches, and gaps. Initially, the biological and computational foundations of the problem are described, highlighting its relevance in bioinformatics as a cornerstone for understanding genetic instructions.

Subsequently, the effectiveness of each method is evaluated through practical experiments with various input instances. The **Dynamic Programming** approach is explored as the method capable of guaranteeing the global optimality of the score, although it presents a higher polynomial computational cost. In contrast, the **Greedy Algorithm** is analyzed from the perspective of temporal efficiency, demonstrating significantly faster execution times, yet without the guarantee of finding the maximum score in all alignment scenarios.

Finally, the implications of the obtained results are discussed, focusing on the trade-off between execution time and the optimality of the generated alignments. The critical analysis highlights how different paradigms can be chosen depending on resource constraints and the precision required in real-world genomic analysis applications.



**Keywords:** Bioinformatics; Dynamic Programming; Greedy Algorithm; DNA Alignment; Computational complexity; Optimization.

## SUMÁRIO

**1. INTRODUÇÃO.....5**
**2. METODOLOGIA.....6**
2.1 Algoritmo de Programação Dinâmica.....6
2.1.1 Obtenção dos Comprimentos das Sequências.....6
2.1.2. Alocação da Matriz de Pontuação.....6
2.1.3. Inicialização das Condições de Borda.....6
2.1.4. Preenchimento da Matriz com Escolha do Máximo.....7
2.1.5. Obtenção da Pontuação Final.....8
2.1.6. Exibição do Consumo de Memória.....9
2.1.7. Liberação da Memória e Retorno.....9
2.1.8. Garantia da Solução Ótima.....9
2.2. Implementação do Algoritmo Guloso.....9
2.2.1. Inicialização das Variáveis de Controle.....10
2.2.2. Percorrimento das Sequências com Escolha Gulosa.....10
2.2.3. Tratamento dos Caracteres Restantes.....12
2.2.4. Exibição do Consumo de Memória.....12
2.2.5. Retorno da Pontuação.....13
2.2.6. Análise da Abordagem Gulosa.....13
**3. RESULTADOS.....14**
3.1. Análise de Desempenho.....14
3.2. Tabela de Pontuação.....16
4. ANÁLISE CRÍTICA E CONCLUSÃO.....18
5. DIVISÃO DE TAREFAS E CONTRIBUIÇÕES.....20
**REFERÊNCIAS.....21**

# 1. INTRODUÇÃO

O alinhamento de sequências biológicas é um problema fundamental na bioinformática e na biologia computacional, especialmente no contexto da análise de sequências de DNA, RNA e proteínas. A comparação entre sequências permite identificar similaridades funcionais, evolutivas ou estruturais, sendo amplamente utilizada em tarefas como montagem de genomas, identificação de mutações, estudo de relações filogenéticas e anotação funcional de genes.

Dentre os modelos clássicos para o alinhamento de sequências, destaca-se o problema do pareamento de sequências com inserção de lacunas (gaps), cujo objetivo é maximizar a pontuação de similaridade entre duas sequências, permitindo a inserção de espaços que representam deleções ou inserções evolutivas. Embora sua formulação pareça simples, a obtenção de um alinhamento ótimo exige a exploração de um espaço combinatório exponencial, o que torna inviável a busca por força bruta para sequências longas.

Uma abordagem amplamente adotada para resolver esse problema de forma exata é a programação dinâmica, que garante o alinhamento de pontuação máxima em tempo polinomial, com complexidade $O(n \times m)$, onde $n$ e $m$ são os comprimentos das sequências. No entanto, em cenários que exigem respostas rápidas ou envolvem grandes volumes de dados, métodos mais eficientes, porém aproximados, como os algoritmos gulosos, são frequentemente empregados em detrimento da qualidade da solução.

Neste trabalho, implementou-se e comparou-se dois paradigmas distintos para o problema do pareamento de sequências de DNA: um algoritmo baseado em programação dinâmica, que produz soluções ótimas, e um algoritmo guloso, que busca soluções aproximadas com menor custo computacional. São analisados o tempo de execução, o consumo de memória e a pontuação de alinhamento obtida por cada método em um conjunto de instâncias fornecido. A partir dos resultados experimentais, discute-se o trade-off entre eficiência e qualidade da solução, evidenciando as vantagens e limitações de cada abordagem no contexto prático da bioinformática.

## 2. METODOLOGIA

## 2.1 Algoritmo de Programação Dinâmica

O algoritmo de programação dinâmica implementado neste trabalho utiliza uma abordagem de preenchimento de matriz para resolver o problema de alinhamento de sequências de DNA. A função *programacao_dinamica(char *s1, char *s2)* foi desenvolvida para garantir a máxima pontuação possível no pareamento entre duas sequências, considerando o sistema de pontuação adotado (base pareada (correta) = +2, base não pareada = -1, lacuna (gap) = -2).

### 2.1.1 Obtenção dos Comprimentos das Sequências

Inicialmente, o algoritmo obtém os comprimentos das duas sequências de entrada utilizando a função *strlen()*:

```
int n = strlen(s1);
int m = strlen(s2);
```

Esses valores são utilizados para definir as dimensões da matriz de programação dinâmica.

### 2.1.2. Alocação da Matriz de Pontuação

O algoritmo aloca dinamicamente uma matriz bidimensional de tamanho (n+1) × (m+1) para armazenar as pontuações parciais. Para otimizar o acesso à memória e reduzir a fragmentação, a matriz é armazenada em um vetor linear (array unidimensional), onde cada célula é acessada pelo índice *i * (m+1) + j* . O tamanho total da memória alocada é calculado como:

```
size_t memoria_matriz = (size_t)(n + 1) * (m + 1) * sizeof(int);
int *matriz = malloc(memoria_matriz);
```

Caso a alocação falhe (retorne *NULL*), o algoritmo exibe uma mensagem de erro e retorna INT_MIN para indicar o problema.

### 2.1.3. Inicialização das Condições de Borda

As bordas da matriz representam alinhamentos envolvendo sequências vazias e são preenchidas com penalidades cumulativas de gap:

- **Primeira linha (i = 0):** representa o alinhamento de uma string vazia com os primeiros *j* caracteres de *s2*. Cada caractere não pareado introduz um gap com penalidade -2. O preenchimento é feito sequencialmente:

```
matriz[0] = 0;
for(int j = 1; j <= m; j++) {
matriz[j] = matriz[j - 1] - 2;
}
```

- **Primeira coluna (j = 0):** representa o alinhamento dos primeiros *i* caracteres de *s1* com uma string vazia. Da mesma forma, cada caractere não pareado adiciona -2 pontos:

```
for(int i = 1; i <= n; i++) {
   int atual = i * (m + 1);
   int anterior = (i - 1) * (m + 1);
   matriz[atual] = matriz[anterior] - 2;
}
```

### 2.1.4. Preenchimento da Matriz com Escolha do Máximo

Após a inicialização, o algoritmo percorre as linhas e colunas restantes da matriz utilizando dois laços *for* aninhados, onde *i* varia de *1 a n* e *j* varia de *1 a m*. Para cada célula (i, j), são calculadas três possíveis pontuações:

| Operação | Descrição | Cálculo no Código |
|---|---|---|
| Diagonal | Parear s1[i-1] com s2[j-1] | score_diag = matriz[diag_ant] + (match ? 2 : -1) |
| Cima (Gap em s1) | Inserir gap em s1 | score_cima = matriz[cima] - 2 |
| Esquerda (Gap em s2) | Inserir gap em s2 | score_esq = matriz[esquerda] - 2 |

Os índices para acesso à matriz linear são calculados da seguinte forma:

```
int diag = i * (m + 1) + j;          // célula atual
int diag_ant = (i-1) * (m+1) + (j-1); // diagonal anterior
```

```
int cima = (i-1) * (m + 1) + j;        // célula acima
int esquerda = i * (m + 1) + (j-1);   // célula à esquerda
```

A pontuação do pareamento diagonal é determinada pela função *comparacao_de_base_DNA()*, que retorna *true* para pares complementares (A-T ou C-G) e *false* para mismatches:

```
if (comparacao_de_base_DNA(s1[i-1], s2[j-1])) {
   score_diag = matriz[diag_ant] + 2;  // match
} else {
   score_diag = matriz[diag_ant] - 1;  // mismatch
}
```

Após calcular as três pontuações, o algoritmo seleciona o maior valor entre elas e o armazena na célula atual:

```
int max = score_diag;
if (score_cima > max) max = score_cima;
if (score_esq > max) max = score_esq;

matriz[diag] = max;
```

Essa escolha do máximo em cada célula é o que garante que a matriz armazene, para cada par de prefixos (i, j), a melhor pontuação possível considerando todas as formas de alinhamento até aquela posição.

### 2.1.5. Obtenção da Pontuação Final

Após o preenchimento completo da matriz, a pontuação ótima do alinhamento global está armazenada na última célula, localizada no canto inferior direito da matriz:

```
pontuacao = matriz[n * (m + 1) + m];
```

Este valor representa a máxima pontuação possível para o alinhamento entre as duas sequências de DNA.

### 2.1.6. Exibição do Consumo de Memória

O código também implementa a exibição da quantidade de memória utilizada pelo algoritmo, somando o tamanho da matriz alocada dinamicamente com o espaço ocupado pelas variáveis locais (aproximadamente 7 inteiros):

```
printf("Memória utilizada (Programação Dinâmica): %zu bytes\n", memoria_matriz + sizeof(int)*7);
```

### 2.1.7. Liberação da Memória e Retorno

Por fim, a memória alocada para a matriz é liberada com free(matriz) para evitar vazamentos de memória, e a pontuação calculada é retornada.

### 2.1.8. Garantia da Solução Ótima

A implementação descrita garante a máxima pontuação possível devido a três propriedades fundamentais:

1. **Subestrutura ótima:** a melhor solução para o problema (i, j) depende apenas das melhores soluções para os subproblemas menores (i-1, j-1), (i-1, j) e (i, j-1).
2. **Sobreposição de subproblemas:** os mesmos subproblemas são reutilizados várias vezes ao longo do preenchimento da matriz, e o armazenamento em tabela evita recálculos desnecessários.
3. **Exploração completa do espaço de soluções:** ao preencher toda a matriz, o algoritmo avalia todas as combinações possíveis de pareamentos e gaps, sem ignorar nenhuma solução viável.

Dessa forma, a programação dinâmica garante que a pontuação retornada seja a melhor possível para o alinhamento entre as duas sequências de DNA, de acordo com o sistema de pontuação adotado.

## 2.2. Implementação do Algoritmo Guloso

O algoritmo guloso implementado neste trabalho utiliza uma estratégia de decisão local imediata para construir o alinhamento entre duas sequências de DNA.

A função *guloso(char *s1, char *s2)* foi desenvolvida para produzir uma solução de forma eficiente, porém sem a garantia de que a pontuação obtida seja a máxima possível. O sistema de pontuação adotado é o mesmo utilizado na programação dinâmica (match = +2, mismatch = -1, gap = -2).

A seguir, são descritas detalhadamente as etapas da implementação presentes no código.

### 2.2.1. Inicialização das Variáveis de Controle

Inicialmente, o algoritmo obtém os comprimentos das duas sequências de entrada utilizando a função *strlen()* e define dois índices *i* e *j* para percorrer *s1* e *s2*, respectivamente, ambos iniciando em zero:

```
int i = 0, j = 0;
int n = strlen(s1);
int m = strlen(s2);
```

A variável *pontuacao* é inicializada com zero e acumulará a pontuação do alinhamento ao longo da execução.

### 2.2.2. Percorrimento das Sequências com Escolha Gulosa

O algoritmo percorre as duas sequências simultaneamente enquanto houver caracteres disponíveis em ambas *(i < n && j < m).* Em cada iteração, são calculadas as pontuações das três operações possíveis para a posição atual:

| Operação | Descrição | Pontuação |
|---|---|---|
| Diagonal | Parear s1[i] com s2[j] | (+2) se forem complementares (A-T ou C-G), `-1` caso contrário |

| | | |
|---|---|---|
| Gap em s1 | Inserir gap em *s1*, alinhando *s2[j]* com vazio | -2 |
| Gap em s2 | Inserir gap em *s2*, alinhando *s1[i]* com vazio | -2 |

No código, essas pontuações são calculadas da seguinte forma:

```
int score_diag = comparacao_de_base_DNA(s1[i], s2[j]) ? 2 : -1;
int score_gap_s1 = -2;
int score_gap_s2 = -2;
```

Após calcular as três pontuações, o algoritmo toma a **decisão gulosa**: escolher a operação que oferece a maior pontuação imediata. A lógica de decisão é implementada com estruturas condicionais:

```
if (score_diag >= score_gap_s1 && score_diag >= score_gap_s2) {
    pontuacao += score_diag;
    i++;
    j++;
}
else if (score_gap_s1 >= score_gap_s2) {
    pontuacao += score_gap_s1;
    j++;
}
else {
    pontuacao += score_gap_s2;
    i++;
}
```

A prioridade é dada à operação diagonal (pareamento) sempre que sua pontuação for maior ou igual às opções de gap. Isso significa que mesmo um mismatch (-1) é preferível à inserção de um gap (-2), pois *-1* >= *-2* é verdadeiro.

### 2.2.3. Tratamento dos Caracteres Restantes

Quando uma das sequências chega ao fim, os caracteres restantes da outra sequência não podem ser pareados e são obrigatoriamente alinhados com gaps. O algoritmo trata essa situação com dois laços *while*:

```
while (i < n) {
   pontuacao -= 2;
   i++;
}
while (j < m) {
   pontuacao -= 2;
   j++;
}
```

Cada caractere remanescente adiciona uma penalidade de `-2` pontos à pontuação final.

### 2.2.4. Exibição do Consumo de Memória

Diferentemente da programação dinâmica, o algoritmo guloso não aloca matrizes ou estruturas de dados complexas, utilizando apenas variáveis locais armazenadas na pilha (stack). O código exibe o consumo de memória aproximado considerando essas variáveis:

```
size_t memoria_total = sizeof(int) * 7 + sizeof(long int);
printf("Memória utilizada (Guloso): %zu bytes\n", memoria_total);
```

### 2.2.5. Retorno da Pontuação

Ao final da execução, o algoritmo retorna a pontuação acumulada:

### 2.2.6. Análise da Abordagem Gulosa

A estratégia gulosa implementada apresenta as seguintes características:

| Carasterística | Descrição |
|---|---|
| Complexidade de tempo | *O(n + m)* — linear, pois percorre cada caractere no máximo uma vez |
| Complexidade de espaço | *O(1)* — constante, pois não utiliza estruturas auxiliares significativas |
| Garantia de solução ótima | *Não* — a escolha localmente ótima pode levar a um resultado global subótimo |

O algoritmo toma decisões baseadas apenas nas pontuações imediatas de cada posição, sem considerar o impacto futuro dessas escolhas. Essa abordagem torna o algoritmo extremamente rápido, porém sacrifica a **qualidade da solução** em comparação com a programação dinâmica, que avalia todas as possibilidades antes de decidir.

## 3. RESULTADOS

A avaliação foi dividida em duas métricas principais para evidenciar o *trade-off* dos paradigmas escolhidos: a eficiência computacional (tempo de execução e consumo de memória) e a otimalidade global (pontuação máxima do alinhamento).

### 3.1. Análise de Desempenho

A análise de desempenho computacional confirma as expectativas teóricas atreladas à complexidade estrutural de cada algoritmo. Conforme ilustrado nos gráficos a seguir, há uma clara divergência no comportamento das duas abordagens à medida que o tamanho da entrada (*N*) cresce.

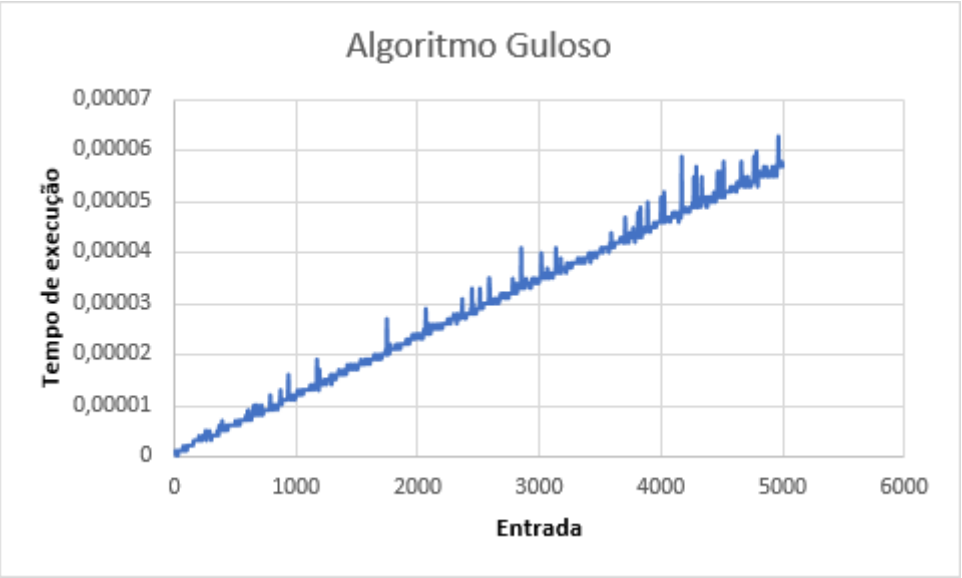


**Figura 1:** Gráfico sobre o tempo de execução em relação a entrada (Algoritmo Guloso)

O Algoritmo Guloso apresentou uma curva de tempo de execução com crescimento linear, refletindo sua complexidade de tempo teórica, que se aproxima de *O(N)*. Como a abordagem gulosa toma decisões imediatas iterando sobre as sequências sem a necessidade de manter um histórico de estados ou realizar *backtracking*, seu consumo de memória manteve-se ínfimo e constante, provando ser uma solução altamente escalável do ponto de vista de recursos computacionais.

**Figura 2:** Gráfico sobre o tempo de execução em relação a entrada (Programação Dinâmica)

Por outro lado, a Programação Dinâmica exibiu um crescimento assintótico polinomial (quadrático), condizente com a sua complexidade de *O(N x M)*. O tempo de processamento escalou significativamente nas instâncias maiores (próximas a 5.000 bases), resultado direto do esforço computacional para o preenchimento da matriz. Da mesma forma, o consumo de memória acompanhou esse crescimento exponencial, evidenciando o elevado custo de armazenamento associado à alocação da matriz de pontuação para o rastreamento do alinhamento ótimo global.

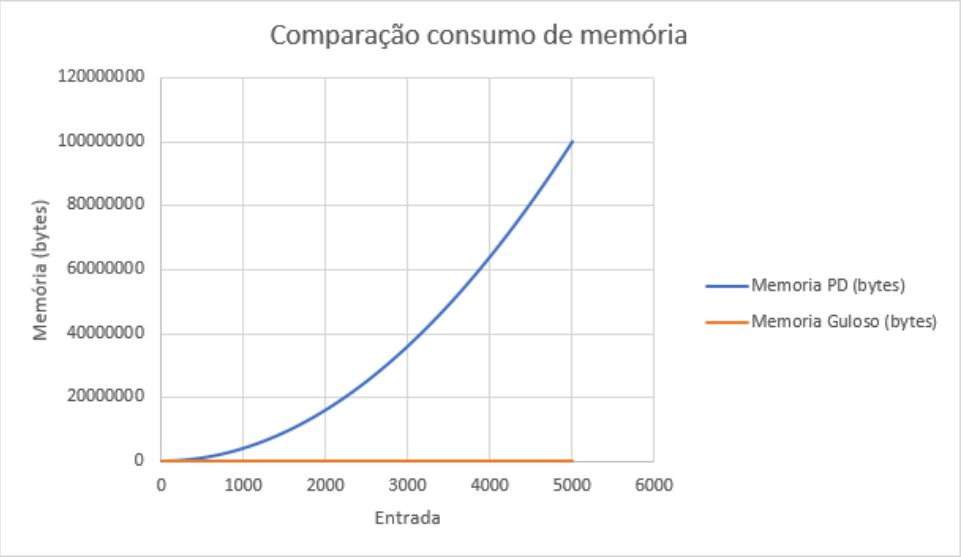


**Figura 3:** Gráfico sobre comparação do consumo de memória entre PD e Algoritmo Guloso

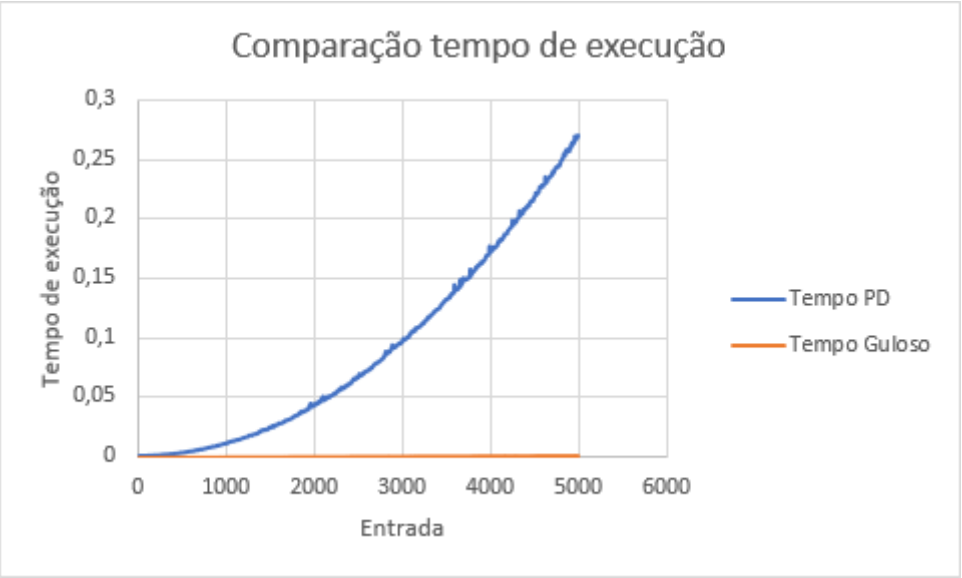


**Figura 4:** Gráfico sobre comparação do tempo de execução entre PD e Algoritmo Guloso

## 3.2. Tabela de Pontuação

Embora o algoritmo guloso tenha se destacado pela eficiência, a análise das pontuações revela o alto preço pago pela sua tomada de decisão estritamente local ("miopia algorítmica"). A tabela abaixo compara o *score* obtido por ambos os métodos.

Para instâncias minúsculas (como *N=5*), ambos os algoritmos obtiveram pontuações semelhantes (*-2*). No entanto, à medida que as sequências aumentaram, ocorreu uma dessincronização drástica na abordagem gulosa. Ao tentar forçar pareamentos imediatos (*matches*), o algoritmo guloso perdeu o alinhamento de blocos inteiros mais à frente, acumulando sucessivas penalidades de *mismatch* e inserções de *gaps*. Isso resultou em uma degradação severa da pontuação, culminando em valores altamente negativos (atingindo pontuações inferiores a *-1200* para instâncias na casa de 5.000 bases).

Por outro lado, o desempenho da **Programação Dinâmica** evidenciou a eficácia da otimização global. O algoritmo foi capaz de encontrar os melhores caminhos possíveis, sacrificando pontuações locais em prol de um alinhamento macro perfeito. A pontuação gerada pela Programação Dinâmica apresentou crescimento positivo consistente e quase linear, alcançando a marca de *2212* pontos para a entrada de 5.000 bases matemáticas.

Esta divergência de milhares de pontos comprova que, para o alinhamento preciso de cadeias de DNA na genômica, a heurística gulosa é inadequada se usada isoladamente para alinhamento global, sendo o custo computacional da programação dinâmica um investimento obrigatório para garantir a integridade dos dados.

Link para tabela com Resultados e Pontuações: https://1drv.ms/x/c/a12ae230f656b143/IQDkbv__F2wkTZcePWAsaoiMAY71RSuPR438_Ztu40GoVO0?e=kXbBLS

## 4. ANÁLISE CRÍTICA E CONCLUSÃO

Este trabalho propôs uma investigação prática sobre o impacto da escolha de paradigmas algorítmicos na resolução do Problema do Pareamento de Sequências de DNA. A implementação e o posterior *benchmarking* dos algoritmos Guloso e de Programação Dinâmica (PD) permitiram não apenas a validação teórica de suas complexidades, mas também uma compreensão profunda do *trade-off* estrutural entre tempo de processamento, consumo de memória e precisão.

A análise crítica dos resultados evidencia que a abordagem Gulosa, embora sedutora por sua execução quase instantânea e custo de memória irrelevante *O(1)* de espaço auxiliar), é fundamentalmente inadequada para o alinhamento global genômico. Os dados comprovaram a vulnerabilidade do método à "miopia algorítmica": ao focar na maximização do ganho imediato (forçando *matches* precoces), o algoritmo perde o contexto global da string. Em sequências maiores, essa dessincronização gerou um efeito cascata de penalidades, resultando em pontuações abissais (negativas). Conclui-se que algoritmos puramente gulosos não são confiáveis para dados biológicos sujeitos a mutações complexas, inserções ou deleções longas.

Por outro lado, o algoritmo de Programação Dinâmica provou ser a única solução metodologicamente segura para garantir a otimalidade global do alinhamento. A capacidade de explorar todas as subestruturas ótimas permitiu que o algoritmo maximizasse a pontuação independentemente do tamanho da entrada. Contudo, essa precisão matemática cobra um custo computacional severo. A complexidade de tempo e espaço $O(N \times M)$ impõe um limite claro de escalabilidade. Durante os testes, o crescimento quadrático do consumo de memória e do tempo de CPU para entradas próximas a 5.000 bases evidenciou que esta abordagem, na sua forma pura, torna-se um gargalo impraticável se aplicada a genomas completos (que possuem bilhões de pares de bases).

Em resumo, o experimento ilustra um dilema clássico na Ciência da Computação aplicado à Bioinformática: a busca pela resposta perfeita versus a restrição de recursos físicos. A prática demonstra que não existe um "algoritmo definitivo". Para análises onde a precisão absoluta é inegociável (como estudos filogenéticos ou identificação de mutações patogênicas em trechos curtos), a

Programação Dinâmica é mandatória. Entretanto, os resultados também justificam por que a indústria atual de bioinformática utiliza soluções híbridas (como o BLAST), que empregam lógicas heurísticas/gulosas para encontrar sementes de alinhamento rapidamente e, apenas nessas janelas promissoras, aplicam o rigor custoso da programação dinâmica.

## 5. DIVISÃO DE TAREFAS E CONTRIBUIÇÕES

O projeto foi executado de forma coletiva, com todos os integrantes participando ativamente das discussões lógicas, testes experimentais e revisões de conteúdo. Para fins de organização, as frentes de liderança e execuções específicas foram distribuídas da seguinte forma:

- **Joaquim Pedro:** Desenvolvimento Principal do Código (Implementação dos paradigmas algorítmicos em C/C++, testes de execução e depuração).
- **Luiz Fernando:** Desenvolvimento Principal do Código (Implementação dos paradigmas algorítmicos em C/C++, testes de execução e depuração).
- **Victória:** Estruturação do Relatório e Análise de Dados (Redação da documentação técnica, consolidação dos resultados na tabela de pontuações e geração dos gráficos gerais de desempenho).
- **Luiz Gabriel:** Estruturação do Relatório e Análise Comparativa (Redação da documentação técnica e elaboração específica dos gráficos de comparação visual entre os algoritmos).
- **Murilo:** Confecção da apresentação (Criação dos slides, síntese dos dados para o público e preparação do material para a defesa em sala de aula).

## REFERÊNCIAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

NEEDLEMAN, Saul B.; WUNSCH, Christian D. A general method applicable to the search for similarities in the amino acid sequence of two proteins. **Journal of Molecular Biology**, v. 48, n. 3, p. 443-453, 1970.

ZIVIANI, Nivio. Projeto de Algoritmos: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.